

Theory Final—Sample Questions—Version of November 1, 2012

Annotated version with some (sometimes rather sketchy) answers and notes.

This is a collection of sample theory final questions. This is just to get some idea of the format of the final and the level of difficulty. These example questions do not guarantee anything about the topics of the questions on your final and they do not guarantee the exact level of difficulty of the questions on your final.

The actual final will consist of five or six questions of equal weight. Your lowest question score won't count.

Note: These questions were adapted from finals that used a different book. It has not been proofread as well as your final will be.

Procedures of the Theory Final

1. The only things that you are allowed to have out during the exam are your cheat sheet, pen(s), pencil(s), an eraser, a ruler, and a simple watch. Please note that you are not allowed to use cellphones, PDAs, etc. as a clock.
2. Please silence your cellphones etc.
3. Do not share your supplies with others and do not talk with others.
4. Use only scrap paper provided by me.
5. Please also hand in your scrap paper and cheat sheet when you are done.
6. Understanding the questions is part of the exam; you are not allowed to ask questions during the exam.
7. As usual, you only have to explain your answer if this is stated in the question.
8. If you use a theorem/construction from your book, please state what you are using where.
9. You may only leave the room after you finish your exam,
10. Your answers should be to the point.

1. (a) Suppose that the alphabet is $\{a, b, c\}$.

For every class of languages in the table, you have to state whether the class is closed under complementation, concatenation, and intersection.

Write “yes” in a box if the class is closed under the operation. Write “no” if the class is not closed under the operation.

closed under	Complementation	Concatenation	Intersection
P	<i>yes</i>	<i>yes</i>	<i>yes</i>
The context-free languages	<i>no</i>	<i>yes</i>	<i>no</i>
The Turing-recognizable languages	<i>no</i>	<i>yes</i>	<i>yes</i>
The decidable languages	<i>yes</i>	<i>yes</i>	<i>yes</i>
The regular languages	<i>yes</i>	<i>yes</i>	<i>yes</i>

- (b) For *one* of the boxes where you answered “no,” give an explicit example that shows why that particular class is not closed under that particular operation. (Clearly state which box you chose.)

To show that a class is not closed under a particular operation, it suffices to give a counterexample. For example, To show that CFLs are not closed under intersection, argue as follows. let $A = \{a^i b^j c^j \mid i, j \geq 0\}$ and let $B = \{a^i b^j c^j \mid i, j \geq 0\}$. Then A and B are CFLs, but $A \cap B = \{a^i b^j c^j \mid i \geq 0\}$, which is not a CFL.

- (c) For *one* of the boxes where you answered “yes,” informally argue why that particular class is closed under that particular operation. (Clearly state which box you chose.)

All “yes” boxes are totally doable.

For example, to show that the decidable languages are closed under complementation, argue as follows. Suppose A is decidable. Then there is an algorithm for A . If you switch the accepts and rejects in this algorithm, you get an algorithm for the complement of A . So, the complement of A is decidable.

2. Consider the following grammar G :

$$S \rightarrow \epsilon \mid 01S \mid 10S \mid 0S1 \mid 1S0 \mid S01 \mid S10 \mid 1S \mid S1$$

(a) Show that this grammar is ambiguous.

You need to give a string and two leftmost derivations or two parse trees for this string. For example, these are two leftmost derivations for the string 1:

$$S \Rightarrow 1S \Rightarrow 1$$

and

$$S \Rightarrow S1 \Rightarrow 1$$

At first glance, it may seem that the language generated by G is

$$L = \{x \in \{0,1\}^* \mid n_0(x) \leq n_1(x)\}.$$

However, this is not true.

(b) For both of the following statements, circle “true” if the statement is true, and circle “false” if the statement is false.

i. $L \subseteq L(G)$ true / false

ii. $L(G) \subseteq L$ true / false

(c) Show that $L(G) \neq L$.

Argue that you can't derive 00111100.

(d) Give a context-free grammar that generates L . (It suffices to give the rules.)

$$S \rightarrow \epsilon \mid SS \mid 0S1 \mid 1S0 \mid S1.$$

3. Let $L = \{a^i b^k c^i d^k \mid i, k \geq 0\}$.

(a) Explain informally why L doesn't seem to be a context-free language.

A PDA that accepts L should push the a 's and the b 's. Then it needs to compare the c 's against the a 's. But the a 's are at the bottom of the stack. To get to the a 's, the b 's have to be popped. But then it is impossible to compare the b 's against the d 's.

(b) Prove that L is not context-free.

Suppose for a contradiction that L is context-free. Let p be the pumping length given by the pumping lemma for context-free languages.

Let $s = a^p b^p c^p d^p$. Then $s \in L$ and $|s| \geq p$. By the pumping lemma for context-free languages, s can be split into five pieces, $s = uvxyz$, such that $|vy| > 0$, $|vxy| \leq p$, and $uv^i xy^i z \in L$ for all $i \geq 0$.

Since $|vxy| \leq p$ and $|vy| \geq 1$, it holds that $vy \in a^+ b^$ or $vy \in b^+ c^*$ or $vy \in c^+ d^*$ or $vy \in d^+$. We will show that all these cases, $uxz \notin L$.*

- i. If $vy \in a^+ b^*$, then uxz contains p c 's and less than p a 's, so $uxz \notin L$.*
- ii. If $vy \in b^+ c^*$, then uxz contains p d 's and less than p b 's, so $uxz \notin L$.*
- iii. If $vy \in c^+ d^*$, then uxz contains p a 's and less than p c 's, so $uxz \notin L$.*
- iv. If $vy \in d^+$, then uxz contains p b 's and less than p d 's, so $uxz \notin L$.*

We have shown that $uxz \notin L$. This is a contradiction.

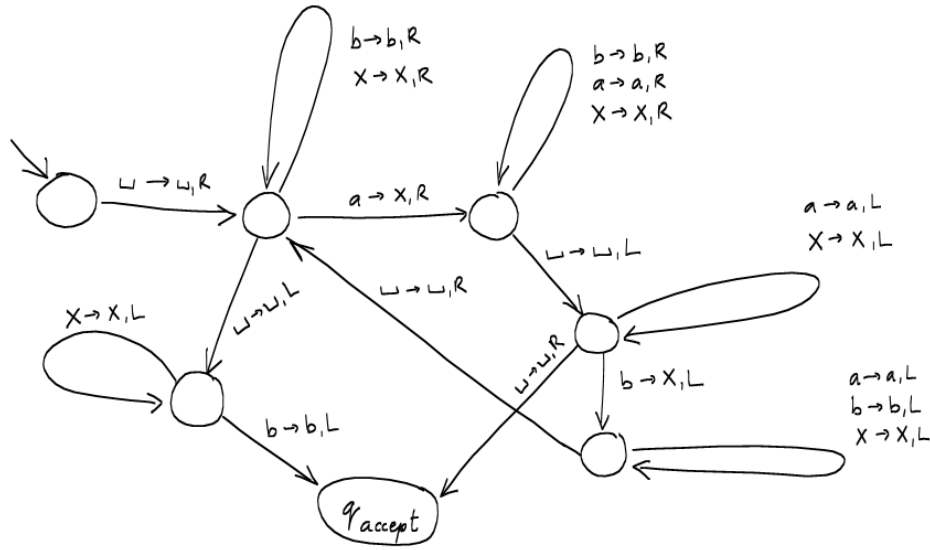
It follows that the assumption that L is context-free is wrong.

So, we have shown that L is not context-free.

4. Give the state diagram of a Turing machine that halts on all inputs and that accepts the language

$$\{x \in \{a, b\}^* \mid n_a(x) \neq n_b(x)\}$$

Note: The TM given here is a little different from Sipser in that it assumes that the tape starts with a blank which is followed by the input.



Give a short explanation to convince me that your Turing machine is correct.

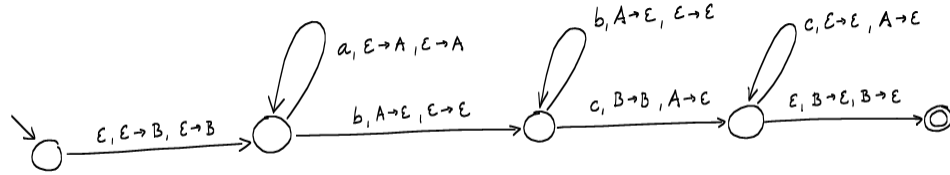
Idea: we go through the string, looking for an 'a' and changing it into 'X', then we go to the end of the string and going backwards we look for a 'b' and change it into 'X'. Thus, we mark off exactly one 'a' and one 'b'. We continue doing this process. If we cannot find an 'a', we have to check that we still have a 'b' to conclude that the number of 'a's and 'b's is not the same. If there is a 'b', we accept, if not, we reject. If we have found an 'a' and we cannot find a 'b', we know that the numbers are not the same and we accept.

5. A 2-PDA is a like a PDA, except that it has two stacks.

(a) Give a language that can be accepted by a 2-PDA, but not by a PDA.

$$\{a^i b^i c^i \mid i \geq 1\}.$$

(b) Draw the state diagram of a 2-PDA that accepts your language from part (a). Explain the notation that you used in your state diagram.



Explanation of notation: $a, b \rightarrow c, d \rightarrow e$ means that we are reading input symbol a , top of the first stack b , top of the second stack d and we are replacing the stack symbols with c on the first stack and e on the second stack. Notice that $a \in \Sigma_\epsilon$, and $b, c, d, e \in \Gamma_\epsilon$.

(c) Give a short, high-level explanation to convince me that your 2-PDA is correct.

Idea: While we read 'a's, we push 'A' onto both stacks. Then for every 'b' we pop 'A' from the first stack and for every 'c' we pop 'A' from the second stack.

6. Consider the following language

$$E_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}.$$

(a) Show that E_{TM} is undecidable.

This follows immediately from Rice's Theorem.

(b) Is E_{TM} in RE (recursively enumerable), in coRE, in both, or in neither? Explain your answer.

E_{TM} is in coRE, but not in RE.

Note that it can not be in both RE and coRE, since then it would be decidable, and we know that it isn't (from part (a)).

It is in coRE, since the following nondeterministic Turing machine S recognizes $\overline{E_{\text{TM}}}$:

$S =$ "On input w :

- i. If w is not of the form $\langle M \rangle$ for M a TM, accept.*
- ii. Otherwise, let $w = \langle M \rangle$ for M a TM.*
- iii. Guess a string x .*
- iv. Run M on input x .*
- v. If M accepts, accept.*
- vi. If M rejects, reject."*

(c) Show that the following language X is undecidable:

$$X = \{\langle M_1, M_2 \rangle \mid M_1 \text{ and } M_2 \text{ are TM's and } L(M_1) \cap L(M_2) = \emptyset\}.$$

Suppose for a contradiction that X is decidable. Let R be a decider for X .

Consider the following TM S to decide E_{TM} .

$S =$ "On input $\langle M \rangle$, where M is a TM:

- i. Run TM R on input $\langle M, M \rangle$.*
- ii. If R accepts, accept.*
- iii. If R rejects, reject."*

Clearly, S decides E_{TM} . This is a contradiction. It follows that X is undecidable.

7. Consider the following decision problem.

$IS-OR-CLIQUE = \{ \langle G, k \rangle \mid G \text{ is a graph, } k \text{ is an integer, and } G \text{ has a clique of size } k \text{ or an independent set of size } k \}.$

(a) Show that $IS-OR-CLIQUE$ is in NP.

Answer 1: $IS-OR-CLIQUE$ is polynomially verifiable. The certificate is the k vertices that form a clique or independent set. It is easy to verify that k vertices form a clique or an independent set.

Answer 2: We have the following NP algorithm for $IS-OR-CLIQUE$.

$S =$ “On input $\langle G, k \rangle$, where G is a graph and k is an integer,

- i. Guess a set of k vertices V .*
- ii. If V is a clique, accept.*
- iii. If V is an independent set, accept.*
- iv. reject.”*

(b) Give an explicit example that shows that the function f defined as $f(\langle G, k \rangle) = \langle G, k \rangle$ is not a polynomial-time many-one reduction from $INDEPENDENT-SET$ to $IS-OR-CLIQUE$.

For example, let G be a clique of size 3 and let $k = 3$. Then G does not have an independent set of size 3, but $\langle G, k \rangle \in IS-OR-CLIQUE$.

(c) Show that $IS-OR-CLIQUE$ is NP-complete.

We'll reduce $INDEPENDENT-SET$ to $IS-OR-CLIQUE$. Define f as follows: $f(\langle G, k \rangle) = \langle H, k + n \rangle$, where n is the number of vertices of G and H is G plus n isolated vertices. We need to show that f is a polynomial-time many-one reduction from $INDEPENDENT-SET$ to $IS-OR-CLIQUE$. It is immediate that f is computable in polynomial time. It is also immediate that if G has an independent set of size k , then H has an independent set of size $k + n$, and thus $\langle H, k + n \rangle \in IS-OR-CLIQUE$. Finally, suppose that $\langle H, k + n \rangle \in IS-OR-CLIQUE$. Since H has $2n$ vertices and at least n of these vertices are isolated vertices, H does not have a clique of size $k + n$. It follows that H has an independent set of size $k + n$. Since H consists of G plus n isolated vertices, it follows that G has an independent set of size k .